

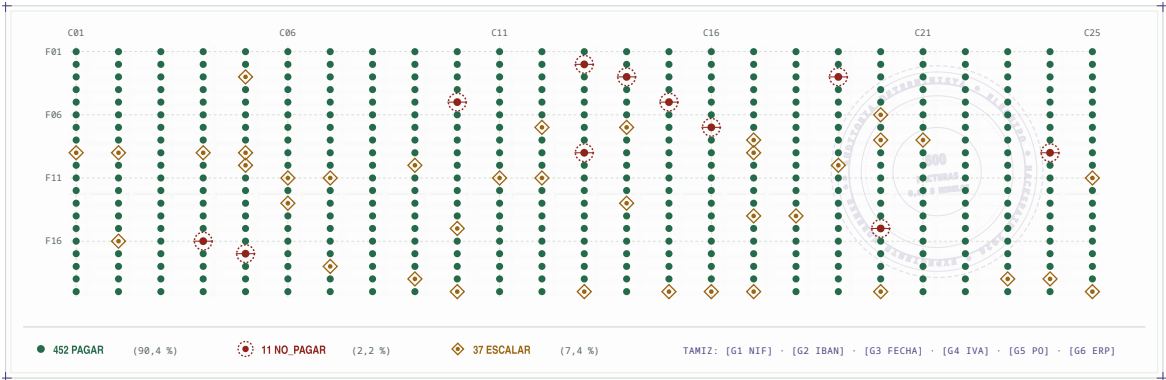
ARQUITECTURA DE REGLAS DETERMINISTAS & AUDITORÍA FORENSE

Otrebla.

Qué facturas pagar, y por qué.

Sistema determinista de seis filtros para quinientas facturas: 452 pagos inmediatos, 11 duplicados detenidos y 37 expedientes con la anomalía identificada para revisión humana.

FIG. 1 · MATRIZ DETERMINISTA DEL LOTE (500 facturas en retícula 25 × 20) 2,06 s · 0,00 \$ MODELOS · 100 % DETERMINISTA



452 90,4 %

PAGAR

2.450.391,46 €

Las seis reglas pasan limpiamente.
Se abonan de forma desatendida sin
riesgo operativo.

11 2,2 %

NO_PAGAR

19.184,92 €

Duplicidad probada frente al
ERP o dentro del lote. Capital
protegido de inmediato.

37 7,4 %

ESCALAR

144.119,95 €

Discrepancias en IBAN, NIF,
fecha o pedido. Expediente
aislado con el dato que no
cuadra ya señalado.

1 Alberto contra las facturas

El trabajo de Alberto es aprobar los pagos. Dicho así, parece darle a un botón. Ojalá. Por cada factura tiene que abrir el PDF, encontrar el pedido en un Excel de catorce hojas, comprobar el NIF y el IBAN del proveedor, sumar base e IVA y entrar en un ERP del 2009 que sigue vivo porque nadie se atreve a cambiarlo, solo para comprobar si ya está pagada. Con una factura es un trabajo tedioso. Con miles, Alberto empieza a sospechar que cuando su padre le dijo que estudiase algo "con salidas", se refería a salidas de emergencia.

Por suerte, nuestro sistema se encarga del trabajo pesado. Alberto sigue teniendo la última palabra, pero ya no son palabrotas. Así funciona:

452 PAGAR · 90,4 % 2.450.391,46 € Las seis reglas pasan. Se paga sin que nadie las mire.	11 NO_PAGAR · 2,2 % 19.184,92 € Pago duplicado demostrado. No es una duda: es un no.	37 ESCALAR · 7,4 % 144.119,95 € Hace falta una persona o un dato corregido.
---	--	--

Lo que el sistema evita

[outcomes.jsonl](#)

Los once **NO_PAGAR** son dinero concreto. Nueve son facturas cuyo asiento en el ERP ya está en PAGADA: pagarlas sería pagar dos veces. Los otros dos son el mismo pedido (**P0-2026-0492**) facturado dos veces dentro del propio lote, algo que sólo se ve mirando el lote entero y no factura a factura.

Por qué escalan las 37

Quince traen un IBAN distinto del registrado: cambio de banco o fraude, 40.063,85 € en juego. Trece facturan un total que no coincide con el pedido ni con el ERP: 69.486,40 €. Seis vienen de un NIF que no está en el maestro de proveedores: 24.623,50 €. Tres tienen una fecha de emisión ilegible o imposible: 9.946,20 €.

El motivo visible es el primero por precedencia, pero el expediente guarda todos los hallazgos. La factura de la página 5 arrastra tres.

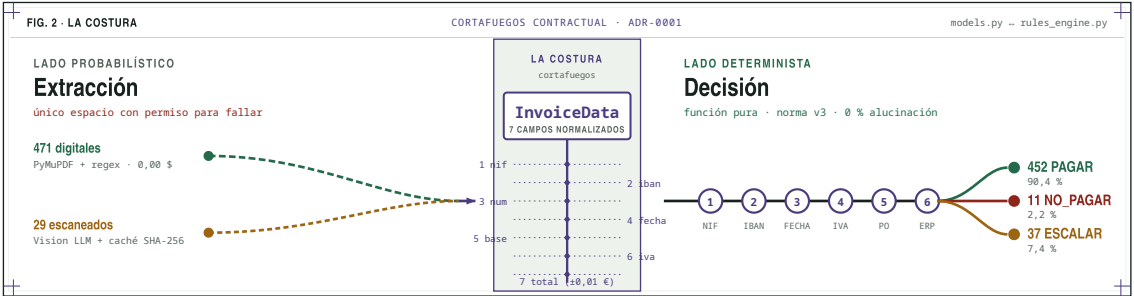
El formato que pide el problema

El trabajo llega por lotes: una carpeta de PDFs, de golpe. Así que el producto es un **proceso por lotes con una sola orden**: eso es lo que se audita, se reejecuta y se mide. Encima hay una **consola de revisión**, no para lanzar el trabajo sino para cerrar las 37 excepciones con el PDF al lado de la regla que falló. Quinientas facturas no necesitan una pantalla; treinta y siete sí.

2 Arquitectura

models.py

Leer un documento es un problema probabilístico; decidir un pago no lo es. Separamos las dos cosas con un contrato de datos en medio, y sólo el lado izquierdo tiene permiso para equivocarse.



Con esto, la pregunta que importa cuando algo sale mal es: ¿leímos mal el documento o aplicamos mal la norma? Y es que son dos capas con dos arreglos distintos

2.1 Las etapas

0	Datos de referencia business_data erp_snapshot	Limpia el Excel (11 proveedores, 516 pedidos) y baja el ERP entero una vez: 516 asientos en 26 páginas. DEJA la foto del ERP en SQLite, con fecha y reintentos. Sin foto, el lote no arranca.
1	Ingesta pipeline._collect_files	Lista la carpeta y descarta repetidos por nombre y por hash del contenido. DEJA la lista del lote con el SHA-256 de cada archivo.
2	Extracción HybridExtractor	Texto incrustado si lo hay; visión si no; línea base si el extractor revienta. DEJA los campos, el método, la latencia, el coste y el fragmento de página del que sale cada campo.
3	Decisión rules_engine	Aplica la norma versionada y busca pedidos repetidos en este lote y en los anteriores. DEJA veredicto, motivo, todos los hallazgos, las 13 comprobaciones y su evidencia.
4	Estado y entregable state · deliverables	Guarda la decisión vigente y la histórica. Escribe el JSONL en un temporal y lo renombra: o está entero o no está. DEJA tres tablas de estado y outcomes.jsonl.
5	Console web/ · Next.js	Cola de revisión, expediente de cada archivo y política editable. DEJA nada propio: lee y escribe a través de la capa de Python. Si se cae, el lote sigue funcionando solo.

Las etapas 0 a 4 son una orden de terminal y dos ficheros SQLite. Ni cola de mensajes ni servicio encendido: quinientos archivos no lo necesitan. Lo que sí necesitan es poder repetirse y explicarse. Lo que cambiaría con otro volumen está en las páginas 6 y 7.

2.2 Estado y reparto del trabajo

state.py

El estado vive en SQLite. El JSONL es el formato de entrega, no la verdad. Esa distinción permite reprocesar sin duplicar.

Bastan tres tablas. `decisions` guarda la decisión vigente de cada archivo, una por `file_id`: reprocesar la sustituye, nunca la duplica. `decision_history` guarda todas las anteriores, inmutables y con su lote, para ver qué cambió al cambiar la norma o el ERP. `runs` guarda cada ejecución: recuentos, segundos, archivos por segundo y coste. La foto del ERP vive al lado, con la fecha de cada descarga.

Código determinista	Modelo de visión	Persona
500 de 500 decisiones	29 de 500 extracciones	37 de 500 revisiones
Lee los 471 PDFs con texto y decide las 500 facturas.	Convierte 29 escaneos en campos, validados contra el esquema.	Resuelve los escalados, corrige campos con una anulación firmada y ajusta la política.
No interpreta imágenes. Si el PDF no trae texto, no lo intenta.	No decide. Su salida entra en las mismas reglas que todo lo demás.	No mira los 452 que cuadran. Ese es el trabajo que se ahorra.

Las correcciones humanas se declaran en un fichero atado al SHA-256 exacto del PDF. Si el documento cambia, la corrección caduca sola y el archivo vuelve a escalar. En el lote 1 hay una (`scan_021.pdf`), anotada como `+human-override`.

2.3 La norma v3, regla a regla

rules_engine.py

La norma vive en código como seis reglas y trece comprobaciones. Cada comprobación tiene un código, y cada código un veredicto por defecto que Alberto puede cambiar sin que nadie toque el motor.

Regla	Contra qué se compara	Códigos y veredicto por defecto
0 Extracción completa	Los campos necesarios para aplicar las reglas.	<code>incomplete_extraction</code> ESCALAR
1 Proveedor e IBAN	Maestro de proveedores del Excel, por NIF.	<code>supplier_not_in_master</code> , <code>iban_mismatch</code> ESCALAR
2 Pedido e importe	Hoja de pedidos: que exista, que sea de ese proveedor y que el total cuadre.	<code>pedido_not_found</code> , <code>pedido_supplier_mismatch</code> , <code>amount_mismatch</code> ESCALAR
3 Aritmética del IVA	La propia factura: base más IVA, y base por tipo.	<code>total_not_base_plus_iva</code> , <code>iva_miscalculated</code> ESCALAR
4 Fecha	La fecha de referencia de la ejecución.	<code>invalid_date</code> , <code>future_date</code> ESCALAR
5 ERP y duplicados	El asiento del ERP, el resto del lote y los lotes anteriores.	<code>already_paid</code> , <code>duplicate_pedido</code> NO PAGAR ; el resto ESCALAR

La tolerancia de importes es de un céntimo. Cuando varios hallazgos apuntan a sitios distintos, la precedencia es **ESCALAR** > **NO PAGAR** > **PAGAR**. El mapa de los 18 códigos a veredictos es un fichero de política editable desde la consola: si Alberto decide que un IBAN distinto es rechazo y no revisión, cambia configuración, no código, y el cambio queda registrado en la ejecución. Los códigos que no se disparan en este lote están cubiertos por las pruebas del motor.

2.4 Seguir una decisión

Esto es lo que el sistema guarda de [2026-0811-B_catering.pdf](#): una factura de Catering Hermanos Pico por 2.920,10 € contra un pedido de 2.795,10 €, decidida en 4,85 ms. No es un resumen escrito para este documento: son las trece comprobaciones tal como salen del motor, cada una con el valor observado, el esperado y la fuente del esperado.

R	Comprobación	Resultado	Observado	Esperado, y de dónde sale
0	Extracción completa	pasa	texto incrustado	campos suficientes Factura
1	Proveedor registrado	pasa	B96233419 Factura	B96233419 Excel · Proveedores · P005
1	IBAN del proveedor	pasa	ES1800815290070001234567 Factura	ES1800815290070001234567 Excel · Proveedores · P005
2	Pedido existente	pasa	PO-2026-0707 Factura	PO-2026-0707 Excel · Pedidos_2026
2	Proveedor del pedido	pasa	P005 Factura y maestro	P005 Excel · Pedidos_2026 · PO-2026-0707
2	Total frente al pedido	no pasa	2.920,10 € Factura	2.795,10 € Excel · Pedidos_2026 · PO-2026-0707
3	Base más IVA	no pasa	2.920,10 € Factura	2.795,10 € Cálculo: 2.310,00 + 485,10
3	Cálculo del IVA	pasa	485,10 € Factura	485,1000 € Cálculo: 2.310,00 × 21 %
4	Fecha no futura	pasa	2026-05-26 Factura	≤ 2026-09-19 Fecha de referencia de la ejecución
5	Asiento en el ERP	pasa	PO-2026-0707 Factura	PO-2026-0707 ERP · AS-70006
5	Total frente al ERP	no pasa	2.920,10 € Factura	2.795,10 € ERP · AS-70006
5	Estado de pago	pasa	PENDIENTE ERP · AS-70006	PENDIENTE Norma de pagos v3
5	Pedido único en el lote	pasa	1 aparición	1 Lote 1 y lotes anteriores

Y lo que acompaña al veredicto

[decisions](#)

Junto a las trece filas queda el veredicto con su motivo principal ([ESCALAR · amount_mismatch](#)), los tres hallazgos completos, la evidencia (P005 · PO-2026-0707 · AS-70006 · PENDIENTE), cómo se leyó el PDF (`embedded_text`, plantilla narrative), la norma aplicada (norma-v3), la latencia (4,85 ms), el coste (0,00 \$) y la hora de ejecución, 2026-09-19T11:20:35Z, con la foto del ERP de esa misma mañana.

Las señales en directo

El proceso emite una línea JSON por suceso: `run_start`, `extracted`, `decided`, `override_skipped`, `data_error`, `data_warnings` y `run_done` con el resumen. La consola consume ese mismo flujo: lo que Alberto ve en pantalla es lo que se guarda en la base de datos. No hay dos caminos.

El trabajo pendiente no es una métrica aparte: es la consulta `result = 'ESCALAR'`. Hoy devuelve 37 filas y 144.119,95 €.

2.5 Escala y coste

runs

Esta página separa dos cosas: lo que medimos en una ejecución real y lo que estimamos a partir de esas medidas. Lo segundo va marcado como estimación.

Medida	Valor	De dónde sale
Lote 1 completo, de carpeta a JSONL	2,055 s	Columna elapsed_s de la ejecución.
Rendimiento	243,3 arch./s	Columna files_per_s, con los 29 escaneos ya en caché.
Extracción por archivo con texto	4,1 ms	Media de 500 filas; mínimo 0,83 ms, máximo 16,64 ms.
Extracción por visión, en frío	5,74 s	Media de los 29 escaneos; mediana 5,22 s, rango de 4,36 a 8,60 s.
Descarga completa del ERP	3,489 s	516 asientos, 26 páginas, 30 peticiones, 3 reintentos.
Coste en modelos de la ejecución	0,00 \$	471 archivos no llaman a ningún modelo; los 29 restantes salieron de caché.

Todo en un portátil: Intel Core i5-12500H, 16 GB, Debian 13, Python 3.13, un solo proceso.

Coste por lote

```
coste_lote = n_escaneos_nuevos × precio_visión
471 digitales × 0 $ + 29 escaneos × 0,0003 $ ≈ 0,0087 $ por los 500 archivos
precio_visión ≈ 0,0003 $/página (0,15 $/M tokens de entrada, 0,60 $/M de salida)
```

El término que falta es el que importa: **los documentos con texto no aparecen en la fórmula**, porque nunca llaman a un modelo. El coste no crece con el número de facturas; crece con el número de *escaneos nuevos*. Y como la caché indexa por contenido, reejecutar un lote entero cuesta cero.

A 100.000 facturas

Con la misma proporción de escaneos (5,8 %), *estimamos* 5.800 llamadas de visión y unos 1,74 \$ en modelos. A esa escala el coste dominante no es el modelo: es el tiempo.

Las 94.200 facturas con texto tardan unos 6,5 minutos en un solo proceso ($94.200 \div 243$). Es trabajo por archivo sin estado compartido: con 8 procesos baja de un minuto. Los 5.800 escaneos son el cuello de botella real: 9,2 horas en serie, 35 minutos con 16 llamadas en paralelo. El techo lo pone la cuota del proveedor, no la máquina. Decidir las 100.000 tarda menos de un minuto: es una función pura en memoria contra dos índices.

Escalar este sistema no es escalar la decisión. Es administrar la cuota de visión y repartir la extracción entre procesos. Las dos cosas viven aisladas en la etapa 2.

2.6 Lo que aguanta y lo que no

Estos son los límites del diseño actual y lo que haríamos en cada caso. Ninguno es una sorpresa: son el precio de elegir lo más pequeño que resolvía el problema.

SQLite admite un solo escritor.

Aparece con más de un proceso escribiendo. La capa de estado es un módulo aislado: se cambia por Postgres sin tocar reglas ni extractores.

El ERP tolera 10 peticiones/s y la sesión caduca a los 15 minutos o 300 usos.

Ya lo esquivamos: una foto completa en 30 peticiones, y las 500 decisiones leen de SQLite. Sin esa foto serían 500 llamadas y una sesión caducada a mitad del lote.

El Excel se carga entero en memoria.

Con 516 pedidos da igual. Con millones, pasa a ser una tabla indexada del mismo almacén de estado.

La cuota del proveedor de visión es el techo del paralelismo.

Se reparte entre varios proveedores, y la caché absorbe las reejecuciones.

Añadir un tipo de archivo nuevo

El punto de extensión es el contrato InvoiceData: un extractor nuevo es una función que devuelve ese objeto, y las reglas no se enteran. Ya existen tres caminos además del PDF (XML de FacturaE y UBL, correos, y texto u hojas de cálculo), detrás de un interruptor de política que hoy acepta sólo .pdf. Aceptar correos en producción es activar ese interruptor y escribir las pruebas del extractor. Ni una línea de la lógica de pago cambia. Eso es lo que compra la costura de la página 3.

Idempotencia

- La clave es el file_id: reprocesar sustituye la decisión, no añade otra.
- La ingesta descarta repetidos por nombre y por hash; la visión se cachea por ese mismo hash.
- Los pedidos de lotes anteriores se leen del historial: una factura repetida entre lotes se ve.
- El JSONL se escribe en un temporal y se renombra: o está entero o no está.

Lo que cubren las pruebas

tests/ · 44 pruebas

44 pruebas en 2,34 s cubren las ramas del motor, el aislamiento entre lotes, el contrato del JSONL, las anulaciones atadas al hash, la degradación sin visión y el bloqueo de una norma no implementada. El motor se prueba sin abrir un PDF.

La precisión de lectura se comprobó a mano sobre 45 documentos: 98,4 % de campos correctos, 100 % en los 30 digitales y 95,2 % en los 15 escaneados. Los dos documentos con algún campo mal son escaneos degradados a propósito, y los dos acabaron escalados. Ningún error de lectura produjo un **PAGAR** indebido.

2.7 Resiliencia y recuperación

Ningún fallo puede convertirse en un pago. Cuando el sistema no sabe, escala; cuando no puede, se para. Nunca rellena el hueco con algo plausible.

El proveedor de visión no responde

Tres intentos con espera creciente (0, 1 y 2 s) y salto a los modelos alternativos. Los 471 digitales y los escaneos en caché no se enteran. Al relanzar, sólo se reintentan los escaneos sin resolver.

Caen todos los proveedores

La extracción se marca como no disponible y la regla 0 no pasa. Esos archivos salen como **ESCALAR** y aparecen en la cola. El resto del lote se decide con normalidad. Se corta la clave en directo y esto es lo que se ve.

El modelo devuelve algo que no es una factura

La salida se valida contra el esquema y se rechaza. Si el extractor revienta, cae a la línea base y lo anota en el método. Nada contaminado entra en las reglas, y un archivo malo no interrumpe el lote: se reintenta y, si insiste, escala.

El ERP falla como suele fallar

ORA-00600: reintenta la misma llamada, hasta 6 veces. 401: vuelve a autenticarse. 429: espera lo que diga la cabecera, y de salida no pasamos de 8,3 peticiones/s sobre un techo de 10. Medido: 3 reintentos en 30 peticiones, sin intervención.

El ERP no está

El lote no arranca. Queda la foto anterior con su fecha, pero decidir con una foto vieja tiene que ser una elección, no un accidente.

Se interrumpe el proceso

Las decisiones ya guardadas y la caché de visión se conservan; el JSONL anterior queda intacto porque se escribe aparte y se renombra al final. Se relanza y los hashes evitan pagar dos veces por lo ya leído.

Se pide una norma que no existe

Error inmediato. No cae a la versión anterior: decidir con la norma equivocada es peor que no decidir.

Qué pasa cuando llega el lote 2

El lote 2 no es una ejecución especial: es otro valor de `--batch`, con su carpeta, su norma y su salida. Las 40 facturas nuevas se procesan solas; el lote 1 no se toca. La norma v4 se aplica en su propio evaluador y la v3 sigue existiendo, así que el lote 1 continúa siendo reproducible. El ERP actualizado se baja como foto nueva, y el lote 2 se niega a arrancar si esa foto no declara la actualización cargada. Si una factura repite un pedido del lote 1, el historial la detecta y sale como **NO_PAGAR**. Y si Alberto cambia un dato de la Caja el domingo, se relanza el lote afectado y «qué cambió y por qué» es una comparación entre dos filas del historial, no una reconstrucción.

Un ejemplo concreto, con la factura de la página 5

Si el asiento **AS-70006** pasara a PAGADA, la regla 5 fallaría por `already_paid` y el veredicto seguiría siendo **ESCALAR**: hay una duda de importe que un pago duplicado no resuelve. Si además cuadrara el importe, pasaría a **NO_PAGAR**.

3 ADRs y trade-offs

De estas tres decisiones depende todo lo anterior. Cada una lleva lo que renunciamos a tener, no sólo lo que ganamos.

ADR-01 · aceptada

Separar la lectura del documento de la decisión de pago

Contexto	Las facturas llegan en seis plantillas y algunas son sólo imagen: leerlas exige modelos probabilísticos. Decidir un pago, en cambio, tiene que ser reproducible y defendible ante una norma escrita.
Alternativas	1. Un modelo lee la factura y dicta el veredicto en la misma llamada. 2. Sólo expresiones regulares, para leer y para decidir. 3. Normalizar todo a un esquema y decidir después con un motor determinista.
Decisión	La tercera. La extracción produce InvoiceData; el motor de reglas consume InvoiceData y no sabe qué es un PDF. Sólo la extracción tiene permiso para ser probabilística.
Consecuencias	Mantener un esquema canónico cuesta coordinación: cambiarlo toca extractores, persistencia y reglas a la vez. Y una lectura equivocada pero plausible sigue pudiendo arrastrar la decisión, así que la evidencia de origen y el escalado siguen siendo necesarios. Aceptamos las dos cosas.
Evidencia	El motor se prueba sin abrir un PDF. La traza de la página 5 dice, para cada comprobación, si el valor discutido viene de la factura, del Excel o del ERP. Ésa es la pregunta que esta separación deja contestar.

ADR-02 · aceptada

Escalar la duda, y reservar el «no» para lo demostrado

Contexto	Pagar una factura dudosa cuesta dinero. Rechazar toda anomalía bloquea pagos legítimos y desgasta la confianza en el sistema. La norma pide revisión humana cuando hay duda razonable.
Alternativas	1. Toda comprobación fallida es NO_PAGAR. 2. Un modelo resuelve los casos ambiguos. 3. ESCALAR para la duda, y NO_PAGAR sólo para el pago duplicado demostrable.
Decisión	La tercera, con precedencia ESCALAR > NO_PAGAR > PAGAR y un mapa de códigos a veredictos configurable. Descartamos la segunda a propósito: un modelo que adjudica casos límite es justo la parte que no podríamos defender ante un auditor.
Consecuencias	La cola de revisión es más larga de lo estrictamente necesario, y la precedencia puede devolver ESCALAR aunque otro hallazgo, por sí solo, ya diga que no se pague. Sale a cuenta si la consola enseña el desajuste exacto. Por eso la consola no es un adorno: es la contrapartida de esta decisión.
Evidencia	37 escalados frente a 11 rechazos en el lote 1. Los 11 se apoyan en un estado PAGADA del ERP o en un pedido repetido dentro del lote, no en una valoración. Y ningún error de lectura detectado en la revisión manual produjo un PAGAR indebido.

ADR-04 · aceptada

Guardar el estado en SQLite, y tratar el JSONL como entrega y no como verdad

Contexto	Hay que saber la decisión vigente de cada archivo, conservar las métricas de cada ejecución y reprocesar cuando cambien la norma o los datos. La consola y el proceso por lotes tienen que ver lo mismo, sin montar infraestructura para un fin de semana.
Alternativas	1. Sólo los ficheros JSONL generados. 2. Ficheros JSON sueltos, uno por archivo. 3. Una base de datos gestionada, con su despliegue. 4. SQLite local, propiedad exclusiva de la capa de Python.
Decisión	La cuarta: runs, decisions con file_id como clave primaria y decision_history inmutable. El JSONL se genera desde ahí. La consola no abre el fichero SQLite: pasa por la misma capa de Python, para que el estado tenga un solo dueño.
Consecuencias	Un solo escritor: en cuanto haya varios trabajadores hay que migrar. Lo asumimos porque la capa de estado está aislada y la migración es local. La tabla de decisiones vigentes no guarda las versiones anteriores de una factura; de ahí el historial. Y una recuperación más fina a mitad de lote exigiría persistir por archivo antes de terminar, que hoy no hacemos.
Evidencia	Las métricas de la página 6 son columnas de la tabla runs, no un cálculo hecho para este documento. La cola de revisión es una consulta. El «qué cambió» entre normas es una comparación entre dos filas del historial.

4 Lo que no podemos demostrar

Los datos son sintéticos y no tenemos la referencia privada.

Podemos demostrar que cada decisión es consistente con la norma y con sus fuentes. No podemos demostrar que coincide con la clave del jurado.

La precisión de lectura sale de 45 documentos etiquetados por una persona.

Es una comprobación por muestreo, no una prueba. El 98,4 % hay que leerlo con ese tamaño de muestra delante.

Un solo nodo y un solo escritor.

Sobra para 500 y aguanta 100.000 en un rato. No sirve para varios trabajadores en paralelo sin migrar el almacén.

El coste de visión es precio de lista.

La ejecución medida costó 0,00 \$ por caché y cuota gratuita. La fórmula sirve; el precio unitario es una estimación.

La recuperación es por lote, no por archivo.

Relanzar cuesta segundos y cero dólares, así que no hemos invertido en un reanudado más fino. A otra escala sería lo primero que arreglaríamos.

5 La consola de revisión

El proceso por lotes ya cumple el encargo: 500 veredictos con su expediente. La mejora que presentamos aparte es lo que pasa *después* del veredicto, que es el trabajo que de verdad ocupa a Alberto.

La cola de revisión con el PDF al lado.

Abre el documento junto a la comprobación que falló, con el valor observado y el esperado. Un escalado deja de ser un aviso y pasa a ser una tarea que se puede cerrar.

La política editable.

Cambiar la tolerancia, la fecha de referencia o el veredicto de un código es un formulario, no un despliegue. Y queda registrado en la ejecución.

La pantalla de indicadores.

Facturas resueltas sin tocar, euros por veredicto, pago doble evitado y proveedores que más escalan. 19.184,92 € y 15 IBANes discrepantes son una conversación con el proveedor, no una métrica.

El historial y el enlace directo.

Cada archivo tiene su URL. «Enséñame la factura 2026-0811-B» es un enlace, incluso en mitad de la defensa.

De nada, Alberto. Ahora lo difícil será explicarle a tu jefe por qué antes tardabas tanto. Puedes dejar el Excel abierto para disimular.
